

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Polimorfismo II

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Significato di un campo riferimento di tipo astratto

- Cosa significa allora un campo (o una variabile locale) riferimento di tipo astratto?
- Supponiamo che **Forma** sia astratta
 - Ovviamente non potremo creare un oggetto di tipo **Forma**
(**Forma** f = new **Forma**())
 - Ma potremo creare istanze di sottoclassi concrete di **Forma** e assegnare il riferimento alla variabile di tipo **Forma**
 - Questo è possibile proprio grazie al **polimorfismo** (es.
Forma f = new **Cerchio**())

La superclasse universale Object

- Tutte le classi in Java ereditano direttamente o indirettamente dalla classe Object
 - Tutti i suoi 11 metodi sono ereditati
- Quando si definisce una classe senza estenderne

```
public class LaMiaClasse  
{  
  
}
```

questo è equivalente a estendere
Object:

```
public class LaMiaClasse extends Object  
{  
  
}
```

I metodi principali della classe Object

<https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html>



I metodi principali della classe Object

Metodo	Descrizione
<code>Object clone()</code>	Restituisce una copia dell'oggetto
<code>boolean equals(Object o)</code>	Confronta l'oggetto con quello in input
<code>Class<? extends Object> getClass()</code>	Restituisce un oggetto di tipo Class che contiene informazioni sul tipo dell'oggetto
<code>int hashCode()</code>	Restituisce un intero associato all'oggetto (per es. ai fini della memorizzazione in strutture dati, hashtable, ecc.)
<code>String toString()</code>	Restituisce una rappresentazione di tipo String dell'oggetto (per default: tipo@codice_hash)

Sovrascrivere il metodo toString

- `toString` è uno dei metodi che ogni classe eredita direttamente o indirettamente dalla classe `Object`
- Non prende argomenti e **restituisce una stringa**
- Chiamato **implicitamente** quando un oggetto deve essere convertito a `String` (es. `System.out.println(o)`)
- L'annotazione **`@Override`** serve a garantire (a noi che scriviamo il codice) che il metodo "sovrascriva" il metodo di una superclasse

```
public class Punto
{
    private int x, y, z;

    public Punto(int x, int y, int z)
    {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public String toString() { return "("+x+","+y+","+z+")"; }
}
```

Sovrascrive
l'implementazione di `toString`
ereditata da `Object`

Sovrascrivere il metodo equals

- Il metodo `equals` viene invocato per confrontare il contenuto di due oggetti
- Per default, se sono lo stesso oggetto (confronto per riferimento), il metodo restituisce `true`

```
public boolean equals(Object o) { return this == o; }
```

- Tuttavia, la classe `Object` non conosce il contenuto delle sottoclassi
 - Per mantenere il “contratto” del metodo è necessario sovrascriverlo

```
public class Punto
{
    private int x, y, z;

    // ...

    @Override
    public boolean equals(Object o)
    {
        if (o == null) return false;
        if (getClass() != o.getClass()) return false;
        Punto p = (Punto)o;
        return x == p.x && y == p.y && z == p.z;
    }
}
```

`getClass()`

- Restituisce il tipo **effettivo** di un oggetto, ossia quello utilizzato per costruirlo.
- Il binding dinamico di Java si basa proprio su questo tipo per stabilire quale metodo chiamare a partire da un riferimento ad un oggetto.

Javadoc sulla simmetria di equals

L'uso di `getClass()` in `equals` garantisce il principio di simmetria richiesto nell'implementazione di `equals`:

`equals`

```
public boolean equals(Object obj)
```

Indicates whether some other object is "equal to" this one.

The `equals` method implements an equivalence relation on non-null object references:

- It is *reflexive*: for any non-null reference value `x`, `x.equals(x)` should return `true`.
- It is *symmetric*: for any non-null reference values `x` and `y`, `x.equals(y)` should return `true` if and only if `y.equals(x)` returns `true`.
- It is *transitive*: for any non-null reference values `x`, `y`, and `z`, if `x.equals(y)` returns `true` and `y.equals(z)` returns `true`, then `x.equals(z)` should return `true`.
- It is *consistent*: for any non-null reference values `x` and `y`, multiple invocations of `x.equals(y)` consistently return `true` or consistently return `false`, provided no information used in `equals` comparisons on the objects is modified.
- For any non-null reference value `x`, `x.equals(null)` should return `false`.

The `equals` method for class `Object` implements the most discriminating possible equivalence relation on objects; that is, for any non-null reference values `x` and `y`, this method returns `true` if and only if `x` and `y` refer to the same object (`x == y` has the value `true`).

Note that it is generally necessary to override the `hashCode` method whenever this method is overridden, so as to maintain the general contract for the `hashCode` method, which states that equal objects must have equal hash codes.

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is the same as the `obj` argument; `false` otherwise.

See Also:

`hashCode()`, `HashMap`

Instanceof e equals

L'uso di `instanceof` in `equals` **non** garantisce il principio di simmetria richiesto nell'implementazione di `equals`.

```
class A {
    int x;

    public boolean equals(Object o) {
        if (!(o instanceof A)) return false;
        A other = (A) o;
        return this.x == other.x;
    }
}

class B extends A {
    int y;

    public boolean equals(Object o) {
        if (!(o instanceof B)) return false;
        B other = (B) o;
        return this.x == other.x && this.y == other.y;
    }
}
```

```
Integer intero = 4;
Object o = intero;
A a = new A();
B b = new B();

a.equals(b); // true (perché b è instanceof A)
b.equals(a); // false (a NON è instanceof B)
```

Joshua Bloch su getClass() vs. instanceof in equals

"The reason that I favor the instanceof approach is that, when you use the getClass approach, you have the restriction that objects are only equal to other objects of the same class, the same run time type. If you extend a class and add a couple of innocuous methods to it, then check to see whether some object of the subclass is equal to an object of the super class, even if the objects are equal in all important aspects, you will get the surprising answer that they aren't equal. In fact, this violates a strict interpretation of the **Liskov substitution principle**, and can lead to very surprising behavior. In Java, it's particularly important because most of the collections (HashMap, etc.) are based on the equals method. If you put a member of the super class in a hash table as the key and then look it up using a subclass instance, you won't find it, because they are not equal."

Un esempio completo con instanceof e downcasting

- Implementiamo un metodo `equals` di confronto tra un oggetto di tipo `SitoWeb` e un altro oggetto qualsiasi:

```
public class SitoWeb
{
    private String url;

    public SitoWeb(String url) { this.url = url; }

    @Override
    public boolean equals(Object o)
    {
        // uso instanceof per garantire che equals
        // funzioni anche sulle sottoclassi
        // (in altre situazioni, e' piu' sensato
        // l'utilizzo di getClass())
        if (!(o instanceof SitoWeb)) return false;

        // downcasting
        return url.equals(((SitoWeb)o).url);
    }
}
```

Metodi e classi final

- Ricordate? con la parola chiave **abstract** obblighiamo i programmatori a implementare certi metodi
- La parola chiave **final** ci permette di fare il contrario: **impedire** ad altri programmatori di:
 - Creare sottoclassi (se specificato di fronte a **class**)
 - Reimplementare (=sovrascrivere) certi metodi (di fronte all'intestazione del **metodo**)

Metodi final: un esempio (1)

- Supponete di creare un conto corrente "sicuro"
 - Accessibile solo tramite la giusta password

```
public class ContoCorrenteSicuro extends ContoCorrente
{
    public boolean controllaPassword(String password)
    {
        // verifica la password
        // ...
    }
}
```

- Tuttavia, estendendo la classe, possiamo "gabbare" l'utente che la userà e accedere noi al conto:

```
public class ContoCorrenteSicuroFidatiDiMe extends ContoCorrenteSicuro
{
    public boolean controllaPassword(String password)
    {
        return true;
    }
}
```

Metodi final: un esempio (2)

- Possiamo evitare questo problema specificando il metodo **final**

```
public class ContoCorrenteSicuro extends ContoCorrente
{
    public final boolean controllaPassword(String password)
    {
        // verifica la password
        // ...
    }
}
```

Nessuno può sovrascrivere
questo metodo!

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.